

486: Predict the Winner

Intuition

The problem is not asking for the `final scores`, but only whether Player `1`'s score is greater than or equal to Player `2`'s.

Therefore, instead of tracking both scores separately, we only need to track their `score difference`.

Example:

Player			
Round	1	2	Diff
1	10	10	0
2	100	100	0

Both outcomes are equivalent because the score difference is `0`, meaning `Player 1` wins.

Likewise:

Player			
Round	1	2	Diff
1	10	5	5
2	100	95	5

These outcomes are also equivalent because the score difference is `5`.

- During `Player 1`'s turn, choosing a value `x` `increases` the score difference by `x`.
 - Conversely, during `Player 2`'s turn, choosing a value `y` `decreases` the score difference by `y`.
-

Essentially, this is a `zero-sum game`:

- Every point `gained` by one player is effectively a `point lost` by the other.

- Therefore, instead of tracking both scores separately, we only need to track their `score difference`.
-

Approach 1: Recursion | Memoization

```
class Solution {
    public boolean predictTheWinner(int[] A) {
        int n = A.length;
        if ((n & 1) == 0) return true;

        int[][] dp = new int[n][n];
        for (int[] r : dp)
            Arrays.fill(r, -1);

        return maxDiff(0, n - 1, A, dp) >= 0;
    }

    private int maxDiff(int i, int j, int[] A, int[][] dp) {
        if (dp[i][j] != -1) return dp[i][j];
        if (i == j) return dp[i][j] = A[i];

        return dp[i][j] = Math.max(
            A[i] - maxDiff(i + 1, j, A, dp),
            A[j] - maxDiff(i, j - 1, A, dp)
        );
    }
}
```

Similar to [877. Stone Game](#)

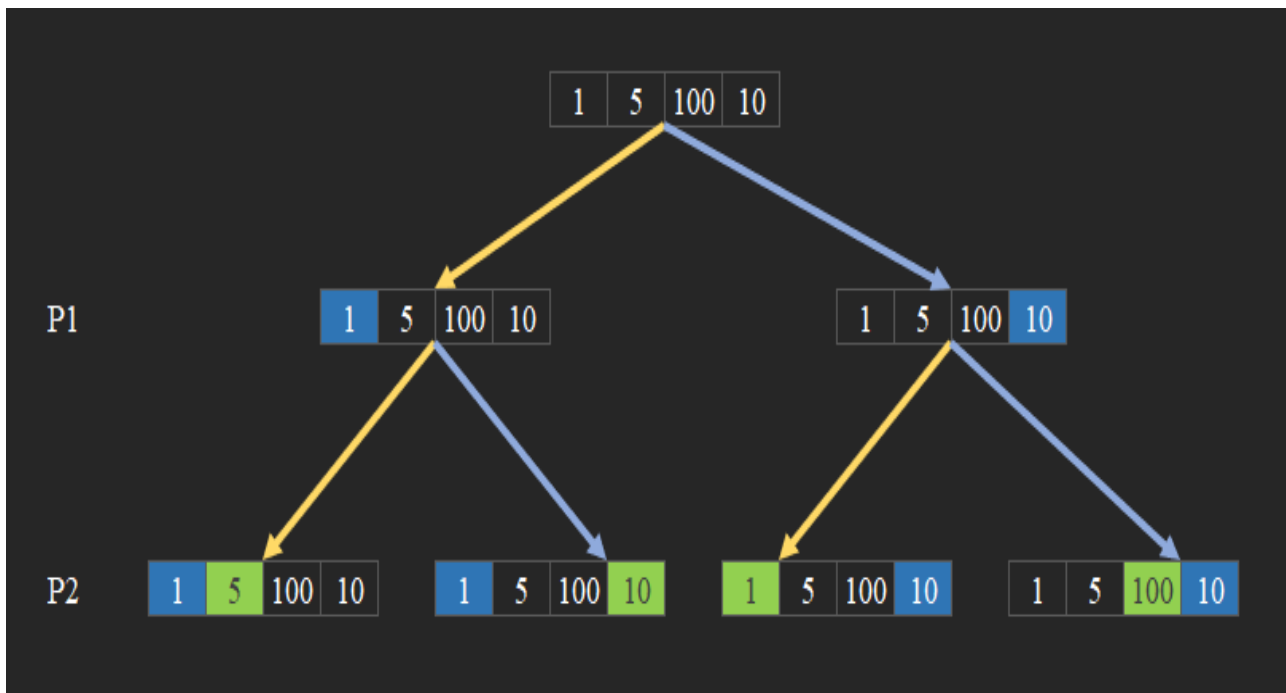
If the `array` has an `even length`, `P1` can always force taking either all even-indexed or all odd-indexed elements. By choosing the parity with the larger total sum, `P1` is guaranteed at least a tie, so the answer is always `true`.

General Case:

Let's define a recursive function `maxDiff(i,j)` using the starting index `i` and ending index `j`.

This function returns the `maximum score` difference that `P1` can achieve over `P2`.

- At each step, we simulate `P1` selecting a number from either end of the current subarray.
- We then recursively evaluate `P2`'s optimal move on the remaining subarray.
- By considering `both` choices, we return the `maximum score` difference that `P1` can guarantee.



Since the players play optimally. Each player always chooses the move that maximizes their own score difference over the opponent.

Suppose it is P1's turn on the subarray nums[i...j].

- The maximum score difference is denoted by maxDiff(i,j).
- P1 has two choices: nums[i] and nums[j].

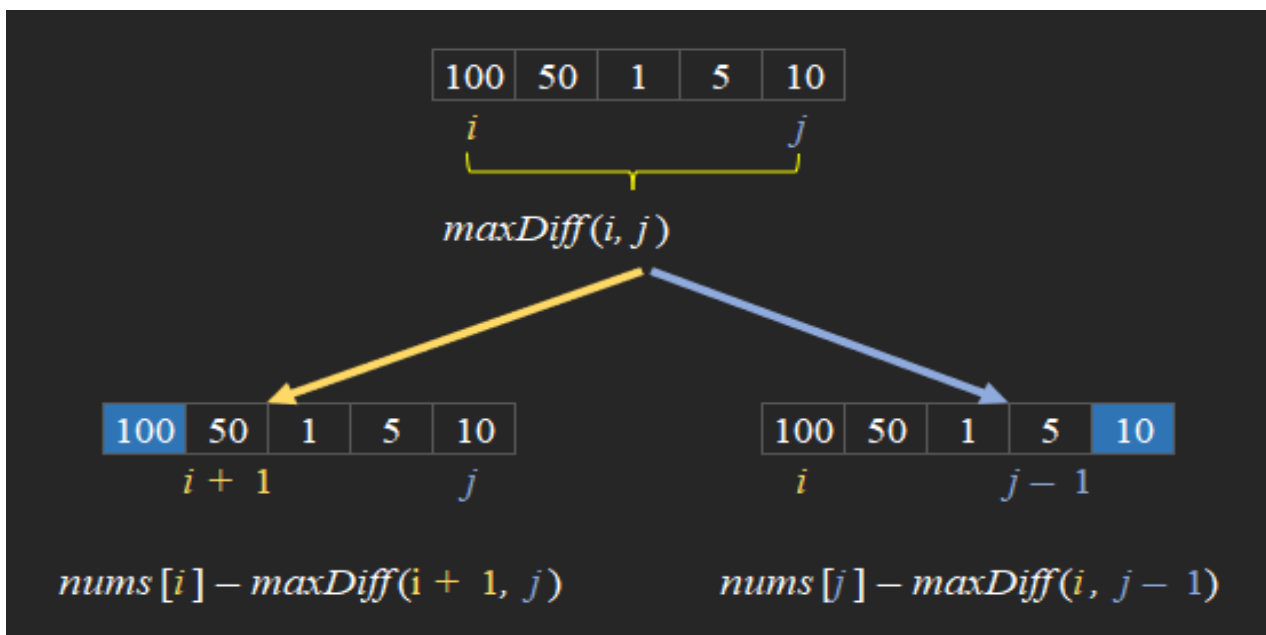
Suppose P1 chooses nums[i]:

- P1 immediately gains nums[i].
- The remaining subarray nums[i+1...j] is then played by P2,
 - where maxDiff(i+1,j) represents the maximum score difference that P2 can achieve over P1.

However, maxDiff(i+1,j) is computed from P2's perspective.

Therefore, every point of advantage for P2 is an equal disadvantage for P1.

Consequently, P2's future advantage must be subtracted from P1's immediate gain.



- Thus, if $\boxed{\text{P1}}$ picks the left value, the resulting score difference is:

$$\text{nums}[i] - \text{maxDiff}(i+1, j)$$

- Likewise, if $\boxed{\text{P1}}$ picks the right value, the resulting score difference is:

$$\text{nums}[j] - \text{maxDiff}(i, j-1)$$

Since $\boxed{\text{P1}}$ plays optimally, the larger of the two values is chosen:

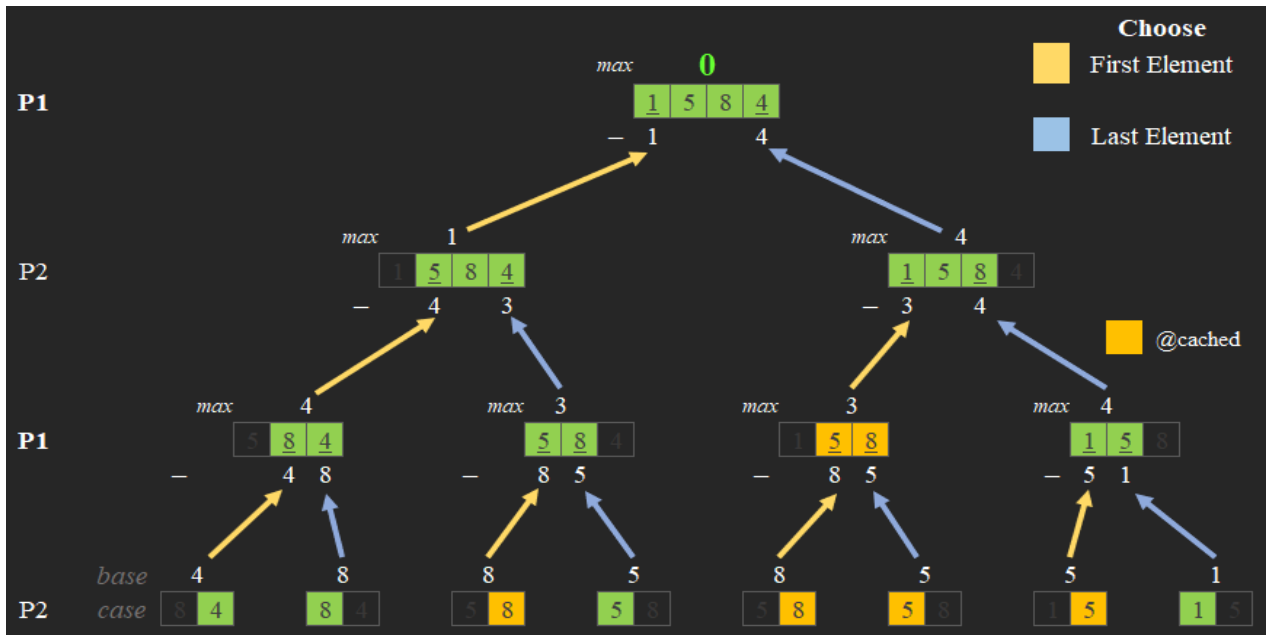
$$\text{maxDiff}(i, j) = \max(\text{nums}[i] - \text{maxDiff}(i+1, j), \text{nums}[j] - \text{maxDiff}(i, j-1)).$$

The recursion continues until the base case $\boxed{i=j}$, where only one number remains. $\boxed{\text{P1}}$ simply takes it, so

$$\text{maxDiff}(i, i) = \text{nums}[i].$$

To avoid recomputing the same $\boxed{\text{subproblem}}$, we can $\boxed{\text{cache}}$ the results of previously computed states.

Example:



DP states can become very large: to simplify the visuals, and have a meaningful examples, we use an array with length of 4.

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$

Approach 2: Bottom-Up DP | Space- Optimized

```
class Solution {
    public boolean predictTheWinner(int[] A) {
        int n = A.length;
        if ((n & 1) == 0) return true;

        int[] dp = new int[n];

        for (int i = n - 1; i >= 0; i--) {
            dp[i] = A[i];
            for (int j = i + 1; j < n; j++)
                dp[j] = Math.max(A[i] - dp[j], A[j] - dp[j - 1]);
        }

        return dp[n - 1] >= 0;
    }
}
```

In recursive approach, each state $\text{maxDiff}(i,j)$ represents the best score difference P1 can achieve over P2 using the subarray $\text{nums}[i\dots j]$.

To compute a larger subarray, we only need the answers of its two smaller subarrays:

- If P1 chooses the left value, we need $\text{maxDiff}(i+1,j)$.
- If P1 chooses the right value, we need $\text{maxDiff}(i,j-1)$.

Since every state depends only on smaller ranges, we can reverse the process:

- Instead of starting from the full array and breaking it down recursively,
- We start from the smallest subarrays and build up to the full array.

The DP state keeps the relation:

$$\text{dp}[i][j] = \text{maxDiff}(i,j).$$

A	11	19	100	2

		END (j)			
		0	1	2	3
START (i)	0	11			
	1		19		
	2			100	
	3				2

		END (<i>j</i>)			
		0	1	2	3
START (<i>i</i>)	0	11			
	1		19		
	2			100	98 S: 100 - 2 = 98 E: 2 - 100 = -98
	3				2

		END (<i>j</i>)			
		0	1	2	3
START (<i>i</i>)	0	11			
	1		19	81 S: 19 - 100 = -81 E: 100 - 19 = 81	-79 S: 19 - 98 = -79 E: 2 - 81 = -79
	2			100	98
	3				2

		END (j)			
		0	1	2	3
START (i)	0	11	8 S: 11 - 19 = -8 E: 19 - 11 = 8	92 S: 11 - 81 = 70 E: 100 - 8 = 92	90 (WIN) S: 11 + 79 = 90 E: 2 - 92 = -90
	1		19	81	-79
	2			100	98
	3				2

Therefore, the recurrence remains:

$$dp[i][j] = \max(nums[i] - dp[i+1][j], nums[j] - dp[i][j-1]).$$

The base case is when the subarray contains only one element.

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$